🧠 Complete Technical Interview Roadmap

DSA + System Design (LLD, HLD, Machine Coding, Fundamentals)

📌 How to Use This Document

This guide is split into two major parts:

1. **DSA Roadmap** — What to learn, in what order, and how to practice
2. **System Design** — LLD, HLD, Machine Coding, and core fundamentals

Work through DSA first (Months 1–4), then layer in System Design (Months 3–6).

PART 1: DSA ROADMAP

🔢 Phase 1 — Foundations (Weeks 1–3)

Before jumping into algorithms, make sure these are solid.

Complexity Analysis

- Time Complexity: O(1), O(log n), O(n), O(n log n), O(n²)
- Space Complexity
- Best / Worst / Average case
- Amortized analysis
- **Practice**: Analyze every solution you write — before and after

Arrays & Strings

Concepts:

- Traversal, insertion, deletion
- Prefix sums and difference arrays
- Kadane's Algorithm (max subarray)
- Dutch National Flag algorithm
- String manipulation: reverse, palindrome, anagram

Must-solve problems:

- Two Sum (HashMap approach)
- Best Time to Buy and Sell Stock
- Product of Array Except Self
- Maximum Subarray (Kadane's)
- Rotate Array
- Valid Anagram
- Longest Common Prefix

Hashing

Concepts:

- HashMap / HashSet internals
- Collision handling
- Frequency counting
- Grouping problems

Must-solve problems:

- Group Anagrams

- Top K Frequent Elements
- Longest Consecutive Sequence
- Subarray Sum Equals K

🔁 Phase 2 — Core Patterns (Weeks 4–7)

Two Pointers

When to use: Sorted arrays, finding pairs, partitioning
Patterns:

- Opposite ends (left + right)
- Same direction (slow + fast)

Must-solve problems:

- Valid Palindrome
- 3Sum
- Container With Most Water
- Remove Duplicates from Sorted Array
- Trapping Rain Water

Sliding Window

When to use: Subarray / substring problems with constraints
Patterns:

- Fixed window size
- Variable window (expand/shrink)

Must-solve problems:

- Maximum Average Subarray
- Longest Substring Without Repeating Characters
- Minimum Window Substring
- Permutation in String

- Longest Repeating Character Replacement

Binary Search

When to use: Sorted arrays, monotonic functions, search space reduction
Patterns:

- Classic binary search
- Search on answer (min/max problems)
- Rotated arrays

Must-solve problems:

- Binary Search (classic)
- Find Minimum in Rotated Sorted Array
- Search in Rotated Sorted Array
- Koko Eating Bananas
- Capacity to Ship Packages
- Median of Two Sorted Arrays

🌳 Phase 3 — Data Structures (Weeks 8–11)

Linked Lists

Concepts:

- Singly / Doubly / Circular
- Fast & slow pointer technique
- Dummy node pattern

Must-solve problems:

- Reverse Linked List
- Merge Two Sorted Lists
- Linked List Cycle (Floyd's algorithm)
- LRU Cache

- Merge K Sorted Lists

Stacks & Queues

Concepts:

- Monotonic stack
- Stack-based DFS
- Queue-based BFS
- Deque for sliding window max

Must-solve problems:

- Valid Parentheses
- Min Stack
- Daily Temperatures (monotonic stack)
- Largest Rectangle in Histogram
- Sliding Window Maximum
- Implement Queue using Stacks

Trees

Concepts:

- Binary Tree traversals (inorder, preorder, postorder) — recursive + iterative
- Binary Search Tree (BST) properties
- Height, diameter, depth
- Level-order (BFS on trees)

Must-solve problems:

- Invert Binary Tree
- Maximum Depth of Binary Tree
- Diameter of Binary Tree
- Level Order Traversal
- Validate BST
- Lowest Common Ancestor
- Binary Tree Right Side View

- Serialize and Deserialize Binary Tree

Heaps / Priority Queue

Concepts:

- Min-heap / Max-heap
- Heap operations: insert O(log n), extract-min O(log n)
- K-th largest/smallest patterns

Must-solve problems:

- Kth Largest Element in Array
- Top K Frequent Elements
- Find Median from Data Stream
- K Closest Points to Origin
- Task Scheduler

🔗 Phase 4 — Graphs (Weeks 12–14)

Graph Fundamentals

- Representation: Adjacency List vs Matrix
- DFS (recursive + iterative)
- BFS (level-order, shortest path)
- Visited array pattern

Core Graph Algorithms

Algorithm	Use Case
BFS	Shortest path (unweighted)
DFS	Connected components, cycle detection
Dijkstra	Shortest path (weighted)
Bellman-Ford	Negative weights
Floyd-Warshall	All-pairs shortest path
Topological Sort	DAG ordering (Kahn's / DFS)
Union-Find (DSU)	Connected components, cycle detection
Prim's / Kruskal's	Minimum Spanning Tree

Must-solve problems:

- Number of Islands
- Clone Graph
- Course Schedule (topological sort)
- Pacific Atlantic Water Flow
- Network Delay Time (Dijkstra)
- Word Ladder (BFS)
- Accounts Merge (Union-Find)
- Cheapest Flights Within K Stops

♻️ Phase 5 — Recursion & Backtracking (Weeks 13–15)

Recursion

Concepts:

- Base case + recursive case
- Call stack visualization
- Memoization vs Tabulation

Must-solve problems:

- Fibonacci (memoized)
- Power of x (fast exponentiation)
- Flood Fill

Backtracking

Template:

```
def backtrack(state, choices):
    if is_solution(state):
        result.append(state)
        return
    for choice in choices:
        make_choice(choice)
        backtrack(new_state, remaining_choices)

        undo_choice(choice)
```

Must-solve problems:

- Subsets / Subsets II
- Permutations / Permutations II
- Combination Sum

- Word Search
- N-Queens
- Sudoku Solver
- Palindrome Partitioning

💡 Phase 6 — Dynamic Programming (Weeks 15–20)

DP Approach

1. Identify if it's a DP problem (overlapping subproblems + optimal substructure)
2. Define the state
3. Write the recurrence
4. Choose top-down (memo) or bottom-up (tabulation)
5. Optimize space if needed

DP Patterns

1D DP

- Climbing Stairs
- House Robber / House Robber II
- Jump Game / Jump Game II
- Coin Change
- Longest Increasing Subsequence

2D DP

- Unique Paths
- Minimum Path Sum
- Edit Distance (Levenshtein)
- Longest Common Subsequence
- 0/1 Knapsack

String DP

- Longest Palindromic Substring
- Longest Palindromic Subsequence
- Regular Expression Matching
- Wildcard Matching

Interval DP

- Burst Balloons
- Matrix Chain Multiplication

DP on Trees / Graphs

- Maximum Sum BST
- Diameter of Binary Tree

🔣 Phase 7 — Advanced Topics (Weeks 18–22)

Tries

- Insert, Search, StartsWith
- Problems: Word Dictionary, Autocomplete, Word Search II

Segment Trees / BITs

- Range sum / range min queries
- Point update, range query

Bit Manipulation

- AND, OR, XOR, NOT, shifts
- Count set bits
- Problems: Single Number, Counting Bits, Reverse Bits

Math

- GCD / LCM (Euclidean)
- Sieve of Eratosthenes (primes)
- Modular arithmetic

📅 DSA Weekly Schedule

Week	Focus
1–2	Arrays, Strings, Hashing
3–4	Two Pointers, Sliding Window
5–6	Binary Search
7–8	Linked Lists, Stacks, Queues
9–11	Trees, Heaps
12–14	Graphs
15	Backtracking
16–20	Dynamic Programming
21–22	Tries, Bits, Advanced

Daily Target: 2–3 problems (1 easy, 1 medium, revisit 1 old)

PART 2: SYSTEM DESIGN

🏗️ System Design Fundamentals

Core Building Blocks

1. Scalability

- **Vertical Scaling:** More RAM/CPU to one machine (has limits)
- **Horizontal Scaling:** More machines (distributed systems)
- **Load Balancer:** Routes traffic — Round Robin, Least Connections, IP Hash
- **Stateless servers** enable easy horizontal scaling

2. Databases

SQL vs NoSQL

	SQL
Schema	Fixed
Scaling	Vertical
ACID	Yes
Examples	MySQL, PostgreSQL
Use when	Complex relations, consistency

Database Techniques

- **Indexing:** B-tree, hash index — speeds up reads, slows writes
- **Sharding:** Split data across DB instances (horizontal partition)
- **Replication:** Master-slave / Master-master for availability
- **Read Replicas:** Offload read traffic from master

3. Caching

Levels:

- Client-side (browser cache)
- CDN (static assets)
- Application-level (in-memory: Redis, Memcached)
- Database query cache

Cache Strategies:

- **Cache-aside (Lazy loading):** App checks cache first, fills on miss
- **Write-through:** Write to cache and DB simultaneously
- **Write-behind:** Write to cache, async write to DB

- **TTL (Time to Live):** Expire stale entries

Cache Invalidation: The hardest problem in CS

- Time-based expiry
- Event-driven invalidation
- Cache-busting on update

4. Message Queues

Why: Decouple producers and consumers, handle spikes, async processing
Tools: Kafka, RabbitMQ, SQS, Pub/Sub
Patterns:

- Point-to-point queue
- Publish-Subscribe (fan-out)
- Dead letter queue (failed messages)
- At-least-once vs at-most-once delivery

5. CDN (Content Delivery Network)

- Caches static assets at edge locations near users
- Reduces latency for global users
- Examples: Cloudflare, AWS CloudFront, Akamai

6. Consistency Models

- **Strong consistency:** All nodes see same data immediately (slower)
- **Eventual consistency:** Nodes converge over time (faster, available)
- **CAP Theorem:** Pick 2 of Consistency, Availability, Partition Tolerance
- **PACELC:** Extension of CAP including latency tradeoffs

7. APIs & Communication

- **REST:** Stateless, HTTP verbs, resource-based
- **GraphQL:** Client defines query shape, single endpoint
- **gRPC:** Binary protocol, efficient for internal microservices

- **WebSockets:** Full-duplex real-time communication
- **Long Polling:** Client polls server repeatedly (older pattern)

8. Rate Limiting

Algorithms:

- Token Bucket (most common)
- Leaky Bucket
- Fixed Window Counter
- Sliding Window Log / Counter

9. Consistent Hashing

Used in distributed caches and DBs to minimize remapping when nodes join/leave. Virtual nodes handle uneven distribution.

🔍 HLD — High Level Design

HLD Framework (Use This in Every Interview)

```
1. Clarify Requirements (5 min)
   - Functional requirements
   - Non-functional: scale, latency, availability, consistency

2. Estimate Scale (2–3 min)
   - DAU (Daily Active Users)
```

```
   - Read/Write ratio
   - Storage estimate
   - Bandwidth estimate

3. High-Level Architecture (5 min)
   - Clients → Load Balancer → App Servers → DB/Cache

4. Deep Dive Components (15 min)
   - Database choice + schema
   - Caching strategy
   - Message queues if async needed
   - CDN for static content

5. Handle Bottlenecks (5 min)
   - Single points of failure
   - Scaling strategies
   - Monitoring and alerting
```

Common HLD Problems & Key Design Points

URL Shortener (TinyURL)

- **Write:** Generate short code (base62), store long→short mapping
- **Read:** Lookup short code, redirect (301 or 302)
- **Scale:** Cache hot URLs in Redis; use NoSQL (Cassandra) for storage
- **Key challenge:** Collision handling, custom aliases, analytics

Instagram / Photo Sharing

- **Storage:** Object store (S3) for images, CDN for delivery
- **Metadata:** SQL for users/follows; NoSQL for posts feed
- **Feed Generation:** Push model (fan-out on write) vs pull model (fan-out on read)
- **Key challenge:** Celebrity problem (push vs pull hybrid)

WhatsApp / Chat System

- **Protocol:** WebSocket for real-time bidirectional messaging
- **Message storage:** Cassandra (high write throughput)
- **Delivery guarantees:** Message queue (Kafka) + ack mechanism
- **Key challenge:** Online presence, group messaging, message ordering

YouTube / Video Streaming

- **Upload:** Chunked upload → transcoding pipeline (multiple resolutions)
- **Storage:** Object store (S3) + CDN for streaming
- **Metadata:** SQL for video metadata, NoSQL for view counts
- **Key challenge:** Adaptive bitrate streaming (ABM), recommendation system

Twitter / News Feed

- **Fan-out on write:** Precompute feed for followers on new tweet (fast read, slow write)
- **Fan-out on read:** Generate feed on request (slow read, fast write)
- **Hybrid:** Fan-out on write for regular users, pull for celebrities
- **Key challenge:** Real-time trending, search

Uber / Ride Sharing

- **Location updates:** Drivers send GPS every few seconds → Redis geospatial index
- **Matching:** Proximity search + surge pricing
- **Trip management:** State machine (requested → accepted → in-progress → completed)
- **Key challenge:** Real-time matching at scale, map routing

Rate Limiter

- **Storage:** Redis for counters (atomic INCR + TTL)
- **Algorithm:** Token bucket or sliding window counter
- **Distributed:** Centralized Redis or gossip protocol for eventual consistency
- **Key challenge:** Distributed rate limiting with low latency

Notification System

- **Components:** Producer (app events) → Message Queue → Workers → Delivery (APNs/FCM/Email)
- **Fanout:** Multiple channels from single event
- **Key challenge:** At-least-once delivery, retry logic, unsubscribe

---

🔧 LLD — Low Level Design

LLD Framework

```
1. Understand requirements
2. Identify core entities (classes/objects)
3. Define relationships (HAS-A, IS-A)
4. Apply SOLID principles
5. Use design patterns where applicable
6. Write clean, extensible code
```

SOLID Principles

Principle	Meaning
Single Responsibility	One class = one reason to change
Open/Closed	Open for extension, closed for modification
Liskov Substitution	Subclass must honor parent's contract
Interface Segregation	Many specific interfaces > one fat interface
Dependency Inversion	Depend on abstractions, not concretions

Key Design Patterns

Creational

- **Singleton:** One instance globally (e.g., DB connection pool)
- **Factory Method:** Let subclass decide which object to create
- **Builder:** Step-by-step construction (e.g., QueryBuilder)

Structural

- **Decorator:** Add behavior without modifying class (e.g., logging wrapper)
- **Adapter:** Bridge incompatible interfaces
- **Facade:** Simplified interface to complex subsystem

Behavioral

- **Observer:** Pub-sub pattern (event listeners)
- **Strategy:** Swap algorithms at runtime (e.g., sort strategies)
- **Command:** Encapsulate action as object (undo/redo)
- **State:** Object changes behavior based on internal state (vending machine)

Common LLD Problems

Parking Lot

Entities: ParkingLot, Floor, Slot, Vehicle, Ticket, ParkingAttendant
Key classes:

- ParkingSlot (type: TWO_WHEELER, FOUR_WHEELER, TRUCK)
- Vehicle (abstract) → Car, Bike, Truck
- ParkingStrategy (interface) → NearestSlotStrategy
- Ticket (slot, vehicle, entry time, price)
- PricingStrategy (hourly, flat rate)

Elevator System

Entities: ElevatorController, Elevator, Button, Request, Display
Key design:

- ElevatorController receives floor requests, dispatches elevators
- Elevator has state machine: IDLE, MOVING_UP, MOVING_DOWN
- Scheduling: LOOK algorithm (scan up then down)
- Request has source floor + direction OR destination floor

Splitwise

Entities: User, Group, Expense, Split, Balance
Key classes:

- Expense → has List<Split> (Equal, Exact, Percentage, Ratio)
- Split (abstract) → EqualSplit, ExactSplit, PercentSplit
- BalanceManager → net balance per user-pair

- Simplify debts: graph-based greedy algorithm

Snake and Ladder Game

Entities: Board, Cell, Player, Dice, Snake, Ladder
Key design:

- Cell has optional Jump (snake or ladder destination)
- Board initializes cells with snakes/ladders
- Game manages turn, win condition
- Dice can be extended for multi-dice

Book My Show (Movie Booking)

Entities: Theater, Screen, Show, Seat, Booking, User, Payment
Key design:

- Seat states: AVAILABLE, LOCKED (temp hold), BOOKED
- Locking seats for X minutes before payment (concurrency critical)
- Booking contains seats, show, user, payment info
- PaymentStrategy → Credit Card, UPI, Wallet

LRU Cache

Python

```
class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.cache = {}  # key → node
        self.head, self.tail = DLinkedNode(), DLinkedNode()
        self.head.next = self.tail
        self.tail.prev = self.head

    def get(self, key): ...
    def put(self, key, value): ...
```

```
    # Move to front on access; evict from tail on overflow
```

ATM Machine

Entities: ATM, Card, Account, Bank, Transaction, CashDispenser
State machine for ATM: IDLE → CARD_INSERTED → PIN_ENTERED → TRANSACTION_SELECTED → PROCESSING → DISPENSING → IDLE

💻 Machine Coding

What Is Machine Coding?

A 60–90 minute coding round where you build a working mini-application from scratch.
Evaluated on:

- Code quality and structure
- OOP design
- Extensibility
- Edge case handling
- Working solution within time

Machine Coding Approach

```
1. Read requirements carefully (5 min)
```

```
   - Note what's explicitly required
   - Note what's NOT required (don't over-engineer)

2. Identify entities and relationships (5 min)

3. Write interfaces / abstract classes first

4. Implement core features (40–50 min)
   - Start with the happy path
   - Add edge cases after it works

5. Dry run with sample input (5 min)


6. Explain your extensibility points (5 min)
```

Common Machine Coding Problems

Cab Booking System (Ola/Uber)

Requirements:

- Add drivers, riders
- Rider requests ride with source/destination
- Available drivers within X km get notified
- Driver accepts → ride starts
- Ride ends → billing calculated

Key classes: RideManager, Driver, Rider, Ride, Location, PricingEngine, MatchingStrategy

Food Ordering System (Zomato/Swiggy)

Requirements:

- Restaurants with menus

- User places order (multiple items)
- Order states: PLACED → ACCEPTED → PREPARING → OUT_FOR_DELIVERY → DELIVERED
- Calculate bill with taxes

Key classes: Restaurant, MenuItem, Order, OrderItem, DeliveryAgent, Cart, BillingService

Inventory Management System

Requirements:

- Add/remove/update products
- Track stock quantity
- Reorder alert when stock < threshold
- Category-based filtering

Key classes: Product, Inventory, Category, StockAlert, InventoryManager

Cache Implementation

Build a cache with pluggable eviction policies (LRU, LFU, FIFO).
Key design: Cache<K,V> with EvictionPolicy interface → LRUEvictionPolicy, LFUEvictionPolicy

Rate Limiter

Build a rate limiter with multiple algorithms.
Key design: RateLimiter interface → TokenBucketRateLimiter, SlidingWindowRateLimiter

Machine Coding Tips

- **Start with models/entities** — get the data right first
- **Use enums** for states and types
- **Prefer composition over inheritance**
- **Write helper methods** — keep methods under 20 lines
- **Don't optimize prematurely** — working code first
- **Handle null/invalid input** at boundaries only

- **Think about thread safety** — mention it even if you don't implement

---

📊 Full Study Timeline

Timeframe	Focus
Month 1	Arrays, Strings, Hashing, T
Month 2	Binary Search, Linked Lists
Month 3	Trees, Heaps, Graphs + Sy
Month 4	Backtracking, DP basics +
Month 5	Advanced DP, Tries, Bits +
Month 6	HLD systems (URL shorter

📚 Recommended Resources

DSA

- **Striver's SDE Sheet** — structured 180-problem sheet
- **Neetcode.io** — pattern-based explanations and roadmap
- **LeetCode** — company-specific filters for targeted prep
- **CSES Problem Set** — for competitive fundamentals

System Design

- **"Designing Data-Intensive Applications"** by Martin Kleppmann — best book on HLD
- **"System Design Interview"** by Alex Xu (Vol 1 & 2) — interview-focused
- **High Scalability Blog** — real-world architecture teardowns
- **Grokking System Design (Educative)** — structured course
- **ByteByteGo (Alex Xu's YouTube)** — visual HLD explanations

LLD

- **Refactoring Guru** — design patterns with examples
- **LeetCode Design tag** — in-platform LLD problems
- **Udit Agarwal (YouTube)** — LLD interview walkthroughs

Document created June 2026 — DSA + System Design Complete Guide